



INVENTIX · TEAM 4

How the layers connect

What actually talks to what, through which file, using which key. The three edges, the two keys, and a restock order traced through all of them.

Source docs/10-connections.md

Companion 01-architecture.md

Generated 3 September 2026

Contents

The shape: a triangle, not a chain

Edge 1 — App to Database, directly

Why only three, when §3.1 allows more

How realtime actually works

Edge 2 — App to Backend

Edge 3 — Backend to Database

One token, two edges

Traced: sending a restock order

Traced: opening the Stocks screen

Which edge is broken

What is connected today

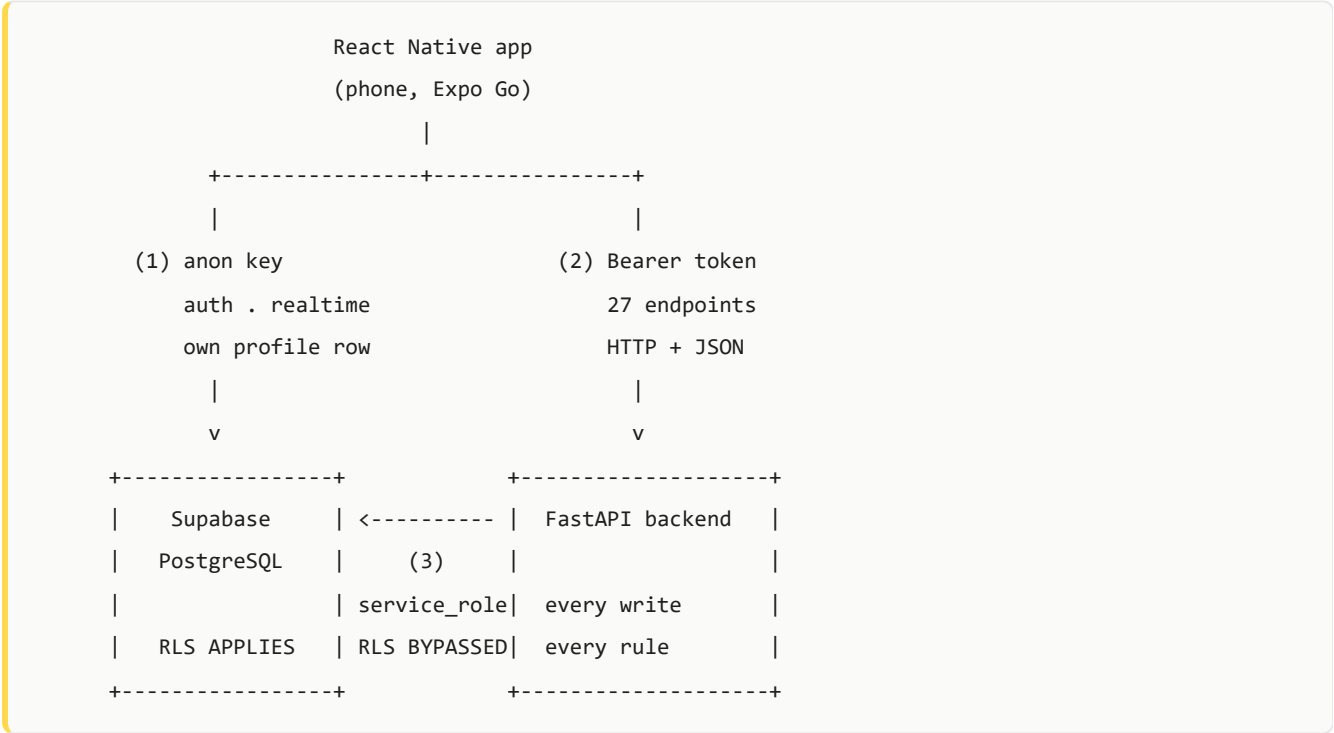
What lights up each edge

The two rules this document exists to protect

What actually talks to what, through which file, using which key. `01-architecture.md` states the rules; this document traces the wiring as it is built.

The shape: a triangle, not a chain

Most systems are a chain — app talks to backend, backend talks to database, and the app never sees the database. InventiX is not that:



The app has **two** outbound connections, not one. Understanding why is most of understanding the architecture.

The reason is in specification §3.1: row level security makes direct reads safe, so the app is allowed to read Supabase itself and to subscribe to realtime. Anything with a *side effect* must go through FastAPI, because that is where the rules live and where the secrets live.

Edge 1 — App to Database, directly

File: `frontend/src/lib/supabase.ts` **Key:** the `anon` public key, from `EXPO_PUBLIC_SUPABASE_ANON_KEY`
Protected by: row level security, on every query

Three uses, and at the moment only three:

Use	Where in the code
Sign in, sign up, sign out, token refresh	<code>stores/authStore.ts</code>
Reading your own <code>profiles</code> row to learn your role at launch	<code>stores/authStore.ts</code> — the only direct table read in the whole app
Realtime subscriptions on <code>orders</code> and <code>stock_items</code>	<code>hooks/useRealtime.ts</code>

Why only three, when §3.1 allows more

§3.1 *permits* the app to read Supabase directly for display. That permission is deliberately not used. Every list, card and number goes through the backend instead.

The reason is one field: `StockItemView.status`. The pie chart and the two list sections both read it, and it is derived from a three-way rule about quantity, threshold and whether an open order exists. If the app ran its own query and derived it independently, the chart could say four items are low while the list below shows three — and both would look correct in isolation. One rule, one place, and that place is

`backend/app/domain/stock.py`.

The cost of that choice is real: the app cannot display anything without the backend running.

How realtime actually works

PostgreSQL writes every change to a write-ahead log for crash recovery. Supabase tails that log and pushes changes to subscribers. So when a supplier advances an order, the customer's screen updates without a refresh.

`useRealtime` only ever triggers a **refetch**. It never applies the change itself:

```
useRealtime('orders', null, orders.refresh);
```

Reads may be direct; conclusions may not. The refreshed data still comes back through the backend, which is what keeps `status` derived in one place.

Edge 2 — App to Backend

File: `frontend/src/api/client.ts` **Base URL:** `EXPO_PUBLIC_API_URL` **Callers:** 11 API files, 27 endpoints

One `fetch()` wrapper. Every request looks like this:

```
POST http://192.168.1.5:8000/customer/ordering/send
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6...
Content-Type: application/json
Idempotency-Key: m8x2k9qp4a7f
```

On a phone, `localhost` is the phone itself. During development `EXPO_PUBLIC_API_URL` must be the **LAN address of the machine running FastAPI**, not localhost. This is the single most common reason a working backend appears to be unreachable.

While that variable is unset, `client.ts` serves fixture data instead and no request is made at all.

Edge 3 — Backend to Database

File: `backend/app/core/supabase.py` **Key:** the `service_role` key, from `backend/.env` **Protected by:** nothing. That is the point.

	Key used	PostgreSQL role	Row level security
App	<code>anon</code>	<code>authenticated</code>	applies — you see only your own rows
Backend	<code>service_role</code>	<code>service_role</code>	bypassed entirely

`BYPASSRLS` is a PostgreSQL role attribute, not a Supabase convention. Every policy in `database/policies/` is simply skipped for the backend.

This is deliberate and necessary — the backend must write `profiles.role`, which no client is allowed to write — but it has a consequence that must be understood: **the backend receives no protection from the database at all**. Every check the database would have made, the backend has to make itself. That is what specification §15.2 requires and what `backend/app/dependencies.py` exists to do.

It is also why the `service_role` key must never appear in the app. A React Native bundle can be unpacked from an installed APK in minutes, and that key would hand the holder the entire database.

One token, two edges

This is the part that ties the triangle together, and it is easy to miss.

- | | |
|--------------------------|---|
| 1. App signs in | Supabase Auth returns a JWT |
| 2. App stores it | AsyncStorage. Survives the app being closed |
| 3. App calls the backend | <code>client.ts</code> pulls that same JWT, attaches it as Bearer |
| 4. Backend decodes it | gets user id, looks up role -> <code>core/security.py</code> |
| 5. App queries Supabase | <code>auth.uid()</code> reads that same JWT -> RLS filters the rows |

One login produces one token, and it satisfies both the backend's authorisation check and the database's row filter. There is no second login, no API key of your own, no session to keep in sync between two systems.

Notice step 3 carefully: the app-to-backend edge **depends on** the app-to-database edge. `client.ts` cannot build a request header without first asking Supabase for the current session:

```
async function authHeader(): Promise<Record<string, string>> {
  if (!supabase) return {};
  const { data } = await supabase.auth.getSession();
  const token = data.session?.access_token;
  return token ? { Authorization: `Bearer ${token}` } : {};
}
```

So if Supabase is unreachable, backend calls do not merely lose their identity — they go out unauthenticated and come back 401.

Traced: sending a restock order

The most consequential action in the application, through every file it touches. This single flow uses all three edges.

#	Layer	What happens
1	Screen	<code>RestockPopup.tsx</code> — the customer taps Send ; the channel chooser opens
2	Screen	<code>handleSend('whatsapp')</code> calls <code>sendOrder(draft, channel, idempotencyKey)</code>
3	API file	<code>api/ordering.ts</code> builds the body and calls <code>request('/customer/ordering/send', ...)</code>
4	Edge 1	<code>client.ts</code> calls <code>supabase.auth.getSession()</code> to get the JWT
5	Edge 2	<code>fetch()</code> sends POST with <code>Authorization</code> and <code>Idempotency-Key</code>
6	Backend	<code>feeds/customer/ordering/routes.py</code> — parses the body against <code>schemas.py</code> , checks the caller is a customer via <code>dependencies.py</code>
7	Backend	<code>service.py</code> re-validates every line against the supplier's listing. The app's validation does not count — the quantity may have changed since the popup opened
8	Backend	<code>core/idempotency.py</code> checks whether this key has been seen
9	Backend	<code>domain/message_builder.py</code> and <code>domain/order_state_machine.py</code> produce the message and confirm <code>requested</code> is a legal starting state
10	Edge 3	<code>core/supabase.py</code> writes with the service key: one <code>orders</code> row, one <code>order_items</code> row per product, and <code>restock_requested = true</code> on each stock item — in one transaction
11	Database	Constraints fire: the <code>status</code> check, the <code>idempotency_key</code> unique index, <code>check (status <> 'rejected' or rejection_reason is not null)</code> , both foreign keys to <code>profiles</code>
12	Database	The <code>reference</code> default pulls <code>DEL-0007</code> from <code>order_reference_seq</code>
13	Backend	Queues the WhatsApp send through <code>integrations/queue.py</code> . If WhatsApp is down the order still exists and the message retries (\$15.4)
14	Response	<code>201 { "order_id": "..." }</code> travels back up
15	Screen	<code>draftStore.closeDraft()</code> , then <code>router.push('/(customer)/delivery/' + orderId)</code>
16	Edge 1, other device	On the supplier's phone, <code>useRealtime('orders')</code> sees the insert arrive through the write-ahead log and calls <code>orders.refresh()</code> — which fetches through edge 2 again

Step 7 is the one worth remembering. The popup already validated the quantity and disabled Send if it was too high — and the backend checks again anyway, because between the popup opening and Send being tapped, another shop may have bought the stock.

Traced: opening the Stocks screen

The read path, which is shorter and shows why the backend sits in the middle of it.

#	Layer	What happens
1	Screen	<code>app/(customer)/stocks/index.tsx</code> mounts and calls <code>useStocks()</code>
2	Hook	<code>hooks/useStocks.ts</code> wraps <code>useAsync(() => listStocks())</code> , which owns loading, error and refresh
3	API file	<code>api/stocks.ts</code> — if <code>useMockData</code> , return the fixture and stop here. Otherwise <code>request('/customer/stocks')</code>
4	Edge 1	Token fetched from the session
5	Edge 2	<code>GET /customer/stocks</code>
6	Backend	<code>routes.py</code> confirms the caller is a customer, calls <code>service.py</code>
7	Edge 3	One query joining <code>stock_items</code> to <code>product_catalog</code> , <code>profiles</code> and <code>supplier_listings</code> . Both supplier joins are <code>LEFT</code> — an item with no preferred supplier is normal and must still appear
8	Backend	<code>domain/stock.py</code> derives <code>status</code> for each row: <code>in_stock</code> , <code>low_stock</code> or <code>restock_requested</code>
9	Backend	<code>schemas.py</code> serialises to <code>StockItemView[]</code>
10	Screen	<code>StockStatusChart</code> reads the summary; <code>StockRow</code> renders each item; the badge colour comes from <code>status</code>

Step 8 is where the design decision from Edge 1 pays off. Because `status` is computed once here, the chart and the list can never disagree.

Which edge is broken

The symptom usually names the edge before you open anything.

Symptom	Edge	Open this
Login fails, but the app otherwise runs	1	<code>lib/supabase.ts</code> , and check the Supabase URL and anon key
Login works; every screen is empty or erroring	2	<code>EXPO_PUBLIC_API_URL</code> — almost always <code>localhost</code> where it should be the LAN IP
Login works; one screen is empty, the rest are fine	2	That feed's <code>service.py</code>
Every API call returns 401 although you are signed in	1 then 2	The token was not attached, or expired. Check the <code>AppState</code> auto-refresh listener in <code>lib/supabase.ts</code>
Every API call returns 403	2	<code>dependencies.py</code> — the role check, not the token
Writes return 500	3	The <code>service_role</code> key, or a constraint refusing the row. Read the Postgres error, not the FastAPI one
Data loads but never updates live	1	<code>hooks/useRealtime.ts</code> , and whether realtime is enabled for that table in Supabase
You can see another shop's data	1	A missing or wrong policy in <code>database/policies/</code> . Stop and run the six-step check in <code>09-database-build.md</code> Part 5

The last row is the only one that is an emergency. Everything above it is a bug; that one is a data breach, and it is invisible until somebody looks.

There is a reason it belongs to edge 1 rather than the backend: the app reads Supabase directly, so a missing policy is not something FastAPI can protect you from. It never sees the query.

What is connected today

Edge	Status
1 — App to Database	Code written, no project. No Supabase project exists, so <code>isSupabaseConfigured</code> is false and the login screen offers demo buttons instead
2 — App to Backend	Code written, nothing to call. <code>EXPO_PUBLIC_API_URL</code> is unset, so all 11 API files return fixtures
3 — Backend to Database	Does not exist. <code>backend/app/</code> is still header-only stubs; the SQL in <code>database/</code> has been verified but not applied anywhere

So nothing is connected to anything. The app on the phone is running entirely on the fixture data in `frontend/src/api/*.ts` .

What lights up each edge

Edge 1 — create the Supabase project, apply `database/` , put the URL and anon key in `frontend/.env` . The demo buttons disappear and registration becomes real. Data is still fixtures.

Edges 2 and 3 together — build the backend and set `EXPO_PUBLIC_API_URL`. These two arrive at the same moment, because the backend is the only thing on edge 2 and it is the only thing that opens edge 3.

There is a trap in that second step. `useMockData` in `client.ts` is a **single global flag**:

```
export const useMockData = !API_BASE_URL;
```

The moment `EXPO_PUBLIC_API_URL` is set, all twelve feeds start calling the backend. If only three are built, the other nine hit endpoints that do not exist. The fix is to replace the boolean with a list of feeds that have gone live, so the migration can happen one feed at a time — cheap to do before the backend exists, annoying halfway through.

The two rules this document exists to protect

Reads may be direct; writes may not. The app may query Supabase and subscribe to realtime, because row level security makes that safe. Anything that changes a quantity, a status or a message goes through FastAPI, because that is where the rules and the secrets live.

The app is never trusted. It may send a request; it may not decide whether it is allowed. Every endpoint re-checks identity and role from the token, and for the direct edge the database re-checks through row level security. Hiding a tab is a convenience for the user, not a security control.